

Hyperspectral Event Inversion Design

<https://gemini.google.com/app/0e8923047dc72a38>

User prompt: I need help designing an inverse problem for hyperspectral PSFs in the event domain. I have calibration PSFs and I need to be able to use those to invert unknown spectra using an inverse method with regularization. EG tikhonov. This is the shell script I used to generate the calibration, the python script I used to perform the calibration, and a video of a event based recording for which I need to decode the spectral response. In the video orange represents polarity 0 and green polarity 1. I do not want you to create any videos. That's not the point, I need help designing an inverse problem.

Response: To recover the unknown spectral distribution $s(\lambda)$ from an event-based vision sensor, we must bridge the continuous differential nature of event camera pixels with the forward physics of a spectrally dispersive Point Spread Function (PSF).

Below is the complete theoretical formulation and numerical solver design for the inverse problem, incorporating **log-rate contrast thresholds**, a **monomode linear system representation**, and **Tikhonov regularization**.

1. Forward Model Formulation

Event cameras generate asynchronous events when the change in log-intensity at pixel (x, y) exceeds a threshold C :

$$\Delta \ln I(x, y, t) = \ln I(x, y, t) - \ln I(x, y, t_0) \approx \pm C$$

Step A: Formulating the Effective Continuous Rate y

Because Tikhonov regularization is a convex linear/quadratic framework, directly fitting discrete, asynchronous spikes (x, y, t, p) is non-convex and ill-conditioned. Instead, we project events into a spatio-temporal activity rate frame $\mathbf{y} \in \mathbb{R}^M$ (where $M = N_x \times N_y$), matching the accumulation strategy in `spectral_correlation.py`:

$$y(x, y) = \frac{1}{\Delta T} \sum_{t_k \in \Delta T} (-1)^{1-p_k} \quad \text{or} \quad y(x, y) = \ln \left(1 + \frac{N_{\text{events}}(x, y)}{\Delta T} \right)$$

Step B: Linearization via Discretized Calibration PSFs

Let the spectrum $s(\lambda)$ be discretized into N_λ spectral bands (e.g., $N_\lambda = 11$ corresponding to 425, 450, ..., 675 nm matching the 25 nm steps in `capture_gaussian_bands.sh`):

$$\mathbf{s} = [s_1 \quad s_2 \quad \dots \quad s_{N_\lambda}]^T \in \mathbb{R}_{\geq 0}^{N_\lambda}$$

For each band k , let $\mathbf{A}_k \in \mathbb{R}^M$ be the vectorized, variance-stabilized calibration PSF frame generated by `spectral_correlation.py`. Stacking these column vectors forms the system design matrix (dictionary matrix):

$$\mathbf{A} = [\mathbf{A}_1 \quad \mathbf{A}_2 \quad \dots \quad \mathbf{A}_{N_\lambda}] \in \mathbb{R}^{M \times N_\lambda}$$

The forward model for the observed event rate frame $\mathbf{y}$ is:

$$\mathbf{y} = \mathbf{A}\mathbf{s} + \boldsymbol{\epsilon}$$

2. Standard Tikhonov Inverse Problem Formulation

We express the inversion as a regularized least-squares minimization problem with non-negativity constraints (since spectral power $s \geq 0$):

$$\hat{\mathbf{s}} = \arg \min_{\mathbf{s} \geq 0} (\|\mathbf{A}\mathbf{s} - \mathbf{y}\|_2^2 + \alpha^2 \|\mathbf{\Gamma}\mathbf{s}\|_2^2)$$

Where:

- **Data Fidelity Term** $\|\mathbf{A}\mathbf{s} - \mathbf{y}\|_2^2$: Measures the L_2 error between measured event spatial activity and modeled PSF superposition.
- **Regularization Parameter** $\alpha > 0$: Controls the trade-off between fitting noise and enforcing smoothness/boundedness.
- **Tikhonov Matrix** $\mathbf{\Gamma}$:
 - **Identity Matrix** ($\mathbf{\Gamma} = \mathbf{I}$): Standard ridge regression (penalizes high energy/outlier spectra).
 - **First-Difference Derivative Matrix** ($\mathbf{\Gamma} = \mathbf{D}_1$): Enforces spectral smoothness across adjacent bands:

$$\mathbf{D}_1 = \begin{bmatrix} -1 & 1 & 0 & \dots & 0 \\ 0 & -1 & 1 & \dots & 0 \\ \vdots & \vdots & \ddots & \ddots & \vdots \\ 0 & 0 & \dots & -1 & 1 \end{bmatrix} \in \mathbb{R}^{(N_\lambda-1) \times N_\lambda}$$

3. Closed-Form & Constrained Numerical Solutions

Case 1: Unconstrained Analytical Solution

If non-negativity is not explicitly bounded in matrix form:

$$\hat{\mathbf{s}} = \left(\mathbf{A}^T \mathbf{A} + \alpha^2 \mathbf{\Gamma}^T \mathbf{\Gamma} \right)^{-1} \mathbf{A}^T \mathbf{y}$$

Case 2: Non-Negative Tikhonov Solution (Recommended)

Since negative spectral energy is physically impossible, we solve the Non-Negative Least Squares (NNLS) system:

$$\hat{\mathbf{s}} = \arg \min_{\mathbf{s} \geq 0} \left\| \begin{bmatrix} \mathbf{A} \\ \alpha \mathbf{\Gamma} \end{bmatrix} \mathbf{s} - \begin{bmatrix} \mathbf{y} \\ \mathbf{0} \end{bmatrix} \right\|_2^2$$

4. Complete Python Implementation

Below is the standalone module that integrates directly with the calibration output of `spectral_correlation.py` to estimate unknown spectra $\mathbf{s}$.

```
import numpy as np
from pathlib import Path
from scipy.optimize import nnls

class HyperspectralEventInverter:
    def __init__(self, psf_frames_dir: str | Path, alpha: float = 1e-2):
        """
        psf_frames_dir: Directory containing calibration .npy files from spectral_correlation.py
        alpha: Tikhonov regularization weight
        """
        self.psf_dir = Path(psf_frames_dir)
        self.alpha = alpha
        self.wavelengths, self.A = self._load_calibration_dictionary()

    def _load_calibration_dictionary(self) -> tuple[np.ndarray, np.ndarray]:
        """Loads calibration PSF numpy arrays and constructs dictionary matrix A."""
        npy_files = sorted(self.psf_dir.glob("*nm_activity.npy"))
        if not npy_files:
            raise FileNotFoundError(f"No calibration .npy frames found in {self.psf_dir}")

        wavelengths = []
        columns = []
        for file in npy_files:
            wl = float(file.stem.split("nm")[0])
            frame = np.load(file).astype(np.float64)
            wavelengths.append(wl)
            columns.append(frame.ravel())

        # Matrix A: shape (M_pixels, N_wavelengths)
        A = np.column_stack(columns)
        return np.array(wavelengths), A

    def _build_tikhonov_matrix(self, num_bands: int, order: int = 1) -> np.ndarray:
        """Constructs 0th order (Identity) or 1st order derivative operator matrix."""
        if order == 0:
            return np.eye(num_bands)
        elif order == 1:
            D = np.zeros((num_bands - 1, num_bands))
            for i in range(num_bands - 1):
                D[i, i] = -1.0
```

```

        D[i, i + 1] = 1.0
    return D
else:
    raise ValueError("Unsupported Tikhonov order.")

def invert(self, y_frame: np.ndarray, reg_order: int = 1) -> dict[str, np.ndarray]:
    """
    Inverts an observed event rate frame to reconstruct the spectral distribution.

    y_frame: Observed 2D activity/event frame matching calibration dimensions.
    reg_order: 0 for ridge regression, 1 for spectral smoothness.
    """
    y_flat = y_frame.astype(np.float64).ravel()
    num_bands = len(self.wavelengths)

    # Build regularization matrix Gamma
    Gamma = self._build_tikhonov_matrix(num_bands, order=reg_order)

    # Augmented System for Non-Negative Tikhonov Least Squares:
    # || [A; alpha*Gamma] * s - [y; 0] ||^2
    A_augmented = np.vstack([self.A, self.alpha * Gamma])
    y_augmented = np.concatenate([y_flat, np.zeros(Gamma.shape[0])])

    # Solve non-negative least squares
    s_hat, residual = nnls(A_augmented, y_augmented)

    # Reconstructed spatial prediction y_rec = A * s_hat
    y_reconstructed = (self.A @ s_hat).reshape(y_frame.shape)

    return {
        "wavelengths": self.wavelengths,
        "spectrum": s_hat,
        "reconstructed_frame": y_reconstructed,
        "residual": residual
    }

# Example Usage:
# inverter = HyperspectralEventInverter("results/spectral_correlation/frames", alpha=0.05)
# result = inverter.invert(observed_evt3_frame, reg_order=1)
# print("Estimated Spectrum:", dict(zip(result["wavelengths"], result["spectrum"])))

```

5. L-Curve Method for Tuning Regularization (α)

To systematically select the optimal parameter α :

1. Solve the system across a logarithmic sweep of $\alpha \in [10^{-5}, 10^2]$.
2. Plot **Residual Norm** $\log \|\mathbf{A}\hat{\mathbf{s}}_\alpha - \mathbf{y}\|_2$ on the x-axis vs. **Solution Regularization Norm** $\log \|\mathbf{T}\hat{\mathbf{s}}_\alpha\|_2$ on the y-axis.
3. Choose α located at the **inflection point (corner)** of the resulting L-shaped curve to achieve maximal noise suppression without over-smoothing the spectral peaks.

User prompt: Yes this is the right approach. DT is 10ms. We aren't worried about reconstructing the frame at this point, just pure spectroscopy. The PSF is designed specifically for spectral encoding at the expense of spatial.

Response: Since spatial position is fixed and your diffractive phase element converts continuous spectral shifts into spatial event distributions across the sensor, we can streamline the inverse problem into a **pure 1D spectral reconstruction task**.

Because spatial resolution is sacrificed to maximize spectral dispersion, the 2D spatial arrangement of events in each 10 ms interval directly acts as a single mixed-spectral fingerprint.

1. Simplified Pure Spectroscopy Model

By vectorizing the full 2D spatial sensor frame into a single measurement vector $\mathbf{y} \in \mathbb{R}^M$ (where $M = N_x \times N_y$), the calibration PSFs form a static spectral transfer matrix $\mathbf{A} \in \mathbb{R}^{M \times N_\lambda}$:

$$\mathbf{y}_{10\text{ms}} = \mathbf{A}\mathbf{s} + \boldsymbol{\epsilon}$$

Where:

- $\mathbf{y}_{10\text{ms}}$: Flat vector of log-transformed event counts ($\log(1 + N)$) or signed activity rates accumulated over $\Delta T = 10$ ms.
- $\mathbf{A} = [\mathbf{a}_{\lambda_1}, \mathbf{a}_{\lambda_2}, \dots, \mathbf{a}_{\lambda_N}]$: Dictionary matrix whose columns $\mathbf{a}_{\lambda_k}$ are the normalized, variance-stabilized 1D spatial activity signatures recorded during calibration for each wavelength band λ_k .
- $\mathbf{s} \geq 0$: Non-negative $N_\lambda \times 1$ spectral power density vector to be estimated.

2. Regularized Non-Negative Formulation

To isolate the spectrum $\mathbf{s}$ without requiring frame synthesis or 2D image reconstruction, we optimize:

$$\hat{\mathbf{s}} = \arg \min_{\mathbf{s} \geq 0} (\|\mathbf{A}\mathbf{s} - \mathbf{y}_{10\text{ms}}\|_2^2 + \alpha^2 \|\mathbf{\Gamma}\mathbf{s}\|_2^2)$$

- **First-Difference Regularization ($\mathbf{\Gamma} = \mathbf{D}_1$)**: Best suited for continuous optical spectra (e.g., broadband illumination, fluorescence emission spectra) to enforce physical continuity across adjacent wavelength bands.
- **Identity Regularization ($\mathbf{\Gamma} = \mathbf{I}$)**: Best suited for discrete line spectra (e.g., laser lines, sparse emissions) to allow sharp monochromatic spikes without over-smoothing.

3. Lightweight Spectroscopy Reconstruction Module

This clean Python implementation bypasses spatial frame re-synthesis, loading the dictionary directly from the `results/` `spectral_correlation/frames` directory generated by `spectral_correlation.py` and returning pure spectral power distributions.

```
import numpy as np
from pathlib import Path
from scipy.optimize import nnls

class EventSpectrometer:
    """Pure spectroscopic decoder for event-based diffractive optics."""

    def __init__(self, calibration_dir: str | Path, alpha: float = 1e-2, reg_type: str = "smoothness"):
        """
        calibration_dir: Directory containing calibration .npy files from spectral_correlation.py
        alpha: Regularization weighting factor
        reg_type: "smoothness" (1st difference for continuous spectra) or "ridge" (L2 for line spectra)
        """
        self.calibration_dir = Path(calibration_dir)
        self.alpha = alpha
        self.reg_type = reg_type
        self.wavelengths, self.A = self._build_spectral_matrix()

    def _build_spectral_matrix(self) -> tuple[np.ndarray, np.ndarray]:
        """Loads calibration PSF arrays and normalizes them into dictionary columns."""
        npy_files = sorted(self.calibration_dir.glob("*nm_activity.npy"))
        if not npy_files:
            raise FileNotFoundError(f"No calibration .npy files found in {self.calibration_dir}")

        wavelengths = []
        columns = []
        for file in npy_files:
            wl = float(file.stem.split("nm")[0])
            frame = np.load(file).astype(np.float64).ravel()

            # Normalize column to unit energy for scale-invariant calibration
            norm = np.linalg.norm(frame)
            if norm > 0:
                frame /= norm

            wavelengths.append(wl)
            columns.append(frame)
```

```

        return np.array(wavelengths), np.column_stack(columns)

def _get_regularization_matrix(self) -> np.ndarray:
    """Generates Tikhonov operator based on expected spectral behavior."""
    n_bands = len(self.wavelengths)
    if self.reg_type == "ridge":
        return np.eye(n_bands)
    elif self.reg_type == "smoothness":
        D = np.zeros((n_bands - 1, n_bands))
        for i in range(n_bands - 1):
            D[i, i] = -1.0
            D[i, i + 1] = 1.0
        return D
    else:
        raise ValueError(f"Unknown reg_type: {self.reg_type}")

def decode_spectrum(self, y_10ms_frame: np.ndarray) -> dict[str, np.ndarray]:
    """
    Inverts a 10ms event accumulation frame to recover pure spectral distribution.

    y_10ms_frame: 2D numpy array of event activity over 10ms
    returns: Dictionary containing 'wavelengths', 'spectrum', and 'residual'
    """
    y_flat = y_10ms_frame.astype(np.float64).ravel()

    # Normalize observation to match dictionary scale
    y_norm = np.linalg.norm(y_flat)
    if y_norm > 0:
        y_flat /= y_norm

    Gamma = self._get_regularization_matrix()

    # Non-negative Tikhonov augmented system: || [A; alpha*Gamma] s - [y; 0] ||^2
    A_aug = np.vstack([self.A, self.alpha * Gamma])
    y_aug = np.concatenate([y_flat, np.zeros(Gamma.shape[0])])

    # Direct non-negative least squares solve for s >= 0
    s_hat, residual = nnls(A_aug, y_aug)

    return {
        "wavelengths": self.wavelengths,
        "spectrum": s_hat,
        "residual": residual,
    }

# Example Usage:
# spec = EventSpectrometer("results/spectral_correlation/frames", alpha=0.01, reg_type="smoothness")
# res = spec.decode_spectrum(evt3_frame_10ms)
# print("Reconstructed Wavelengths (nm):", res["wavelengths"])
# print("Spectral Power Distribution:", res["spectrum"])

```

4. Dynamic Time-Series Decoding

Since your accumulation interval $\Delta T = 10$ ms, you can process a dynamic stream of event data by stepping through 10 ms sliding windows and calling `decode_spectrum()` on each chunk. This produces a **2D Spectrogram** ($N_\lambda \times N_{\text{time_steps}}$), tracking real-time spectral evolution directly in the event domain.
